

Desarrollo de aplicaciones web en el entorno servidor

MF0492 – UF1844

UF1844: DESARROLLO DE APLICACIONES WEB EN EL ENTORNO SERVIDOR (90 horas)

- EL PROCESO DE DESARROLLO DE SOFTWARE
- LA ORIENTACIÓN A OBJETOS
- ARQUITECTURAS WEB
- LENGUAJES DE PROGRAMACIÓN DE APLICACIONES WEB EN EL LADO SERVIDOR





EL PROCESO DE DESARROLLO DE SOFTWARE

INGENIERÍA DEL SOFTWARE

- *La ingeniería de software es el establecimiento y uso de principios fundamentales de la ingeniería con objeto de desarrollar en forma económica software que sea confiable y que trabaje con eficiencia en máquinas reales.*
- La ingeniería de software es una tecnología con varias capas. Cualquier enfoque de ingeniería (incluso la de software) debe basarse en un compromiso organizacional con la calidad por eso el fundamento en el que se apoya la ingeniería de software es el compromiso con la calidad.
- Otro de los fundamentos es el de proceso, ya que define la estructura que debe establecerse para la obtención eficaz de tecnología de ingeniería del software.
- El proceso de software forma la base para el control de la administración de proyectos de software, y establece el contexto en el que se aplican métodos técnicos, se generan productos del trabajo (modelos, documentos, datos, reportes, formatos, etc.), se establecen puntos de referencia, se asegura la calidad y se administra el cambio de manera apropiada.

CICLO DE VIDA

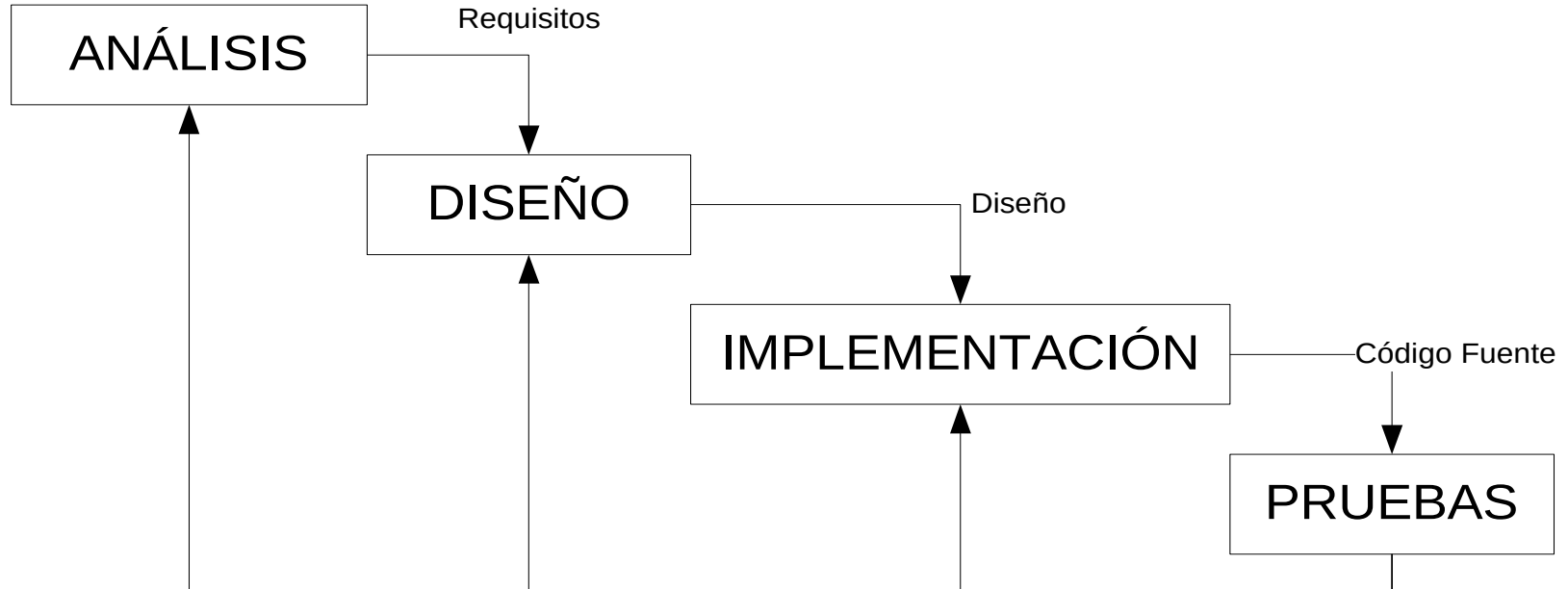
- Según el estándar ISO/IEC 12207 Information Technology / Software Life Cycle Processes
 - *El ciclo de vida de un sistema de información (SI) es el marco de referencia que contiene los procesos, las actividades y las tareas involucradas en el desarrollo, la explotación y el mantenimiento de un producto de software, abarcando la vida del sistema desde la definición de los requisitos hasta la finalización de su uso.*
- Todo ciclo de vida debe cubrir tres objetivos básicos:
 - Definir las actividades a realizar y en qué orden.
 - Asegurar la consistencia con el resto de los sistemas de información de la organización.
 - Proporcionar puntos de control para la gestión del proyecto (calendario y presupuesto).
 - PMBOK

MODELOS DE CICLOS DE VIDA

- **Modelos tradicionales:** Son los de más amplia utilización.
 - Modelo Code And Fix
 - Modelo en cascada.
 - Modelos basados en prototipos:
 - Modelo de construcción de prototipos.
 - Modelo de desarrollo incremental.
 - Modelo de prototipado evolutivo.
- **Modelos alternativos**
 - Modelo en espiral
 - Modelos basados en transformaciones (generar código a partir de unas especificaciones transformándolas por medio de herramientas)
 - Proceso unificado de desarrollo del software.
 - Modelo basado en Desarrollo de Software Basado en Componentes.
 - Modelo basado en Extreme Programming.

MODELO EN CASCADA

- Ha tenido desde los años 70 un papel fundamental en el desarrollo de proyectos software, tanto es así que se le conoce también como modelo clásico del ciclo de vida del desarrollo de software. Fue enunciado por Winston Royce en 1970.
- Este modelo se compone de una serie de fases que se suceden secuencialmente, generándose en cada fase resultados que serán necesarios para iniciar la fase siguiente.



MODELO EN CASCADA

- **1ª Fase - Análisis**
 - Consiste en la recopilación de los requisitos del software.
 - Se debe comprender el ámbito de información del software así como la función, el rendimiento y las interfaces requeridas.
 - Estos requisitos se deben documentar y revisar de tal manera que los entiendan tanto los usuarios como el equipo de desarrollo del software.
 - En esta fase se desarrollará la Especificación de Requisitos Software.
- **2ª Fase - Diseño**
 - A partir de la ERS se descompone y organiza el sistema en componentes que puedan ser desarrollados por separado.
 - El resultado del diseño es la colección de especificaciones de cada elemento componente, definiendo el “como” diseñando las estructuras de datos, la arquitectura del sistema...
 - En esta fase se desarrollará el Documento del diseño del Software que será una descripción de la estructura global del sistema.

MODELO EN CASCADA

- **3ª Fase - Implementación**
 - En esta fase se traduce el diseño a un lenguaje de programación capaz de ser interpretado y ejecutado por el ordenador.
 - El resultado de esta fase será el Código fuente.
- **4ª Fase - Pruebas**
 - Esta fase comienza antes de que el sistema haya sido entregado al usuario, y continúa durante toda la vida útil.
 - Se controlan errores no detectados previamente, se realizan modificaciones, mejoras o ampliaciones solicitadas por los usuarios o debidas a la evolución del entorno.



LA ORIENTACIÓN A OBJETOS

INTRODUCCIÓN A LA OO

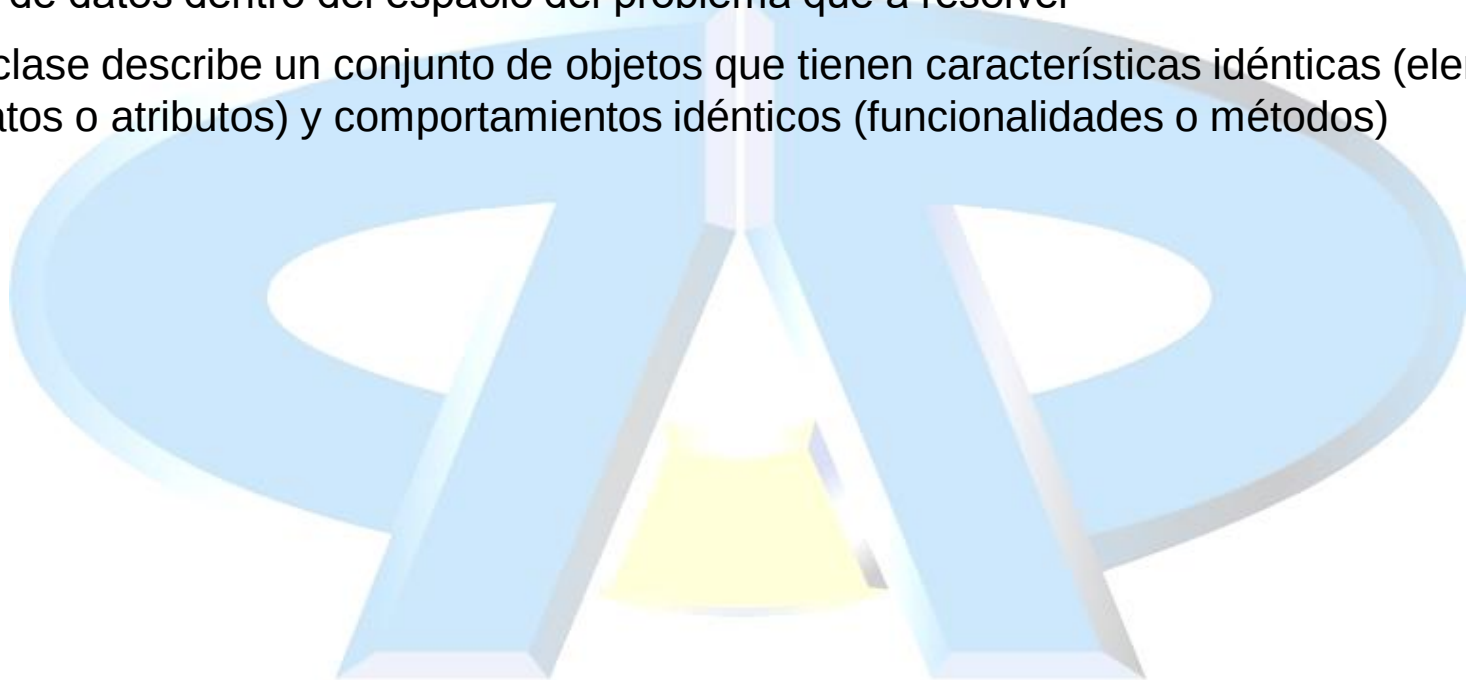
- Paradigma de programación en el que un sistema se representa como un conjunto de objetos que interactúan entre sí
- Características
 - Todo es un objeto
 - Un programa es un conjunto de objetos que se comunican mediante mensajes
 - Cada objeto puede estar compuesto de otros objetos
 - Cada objeto tiene un tipo (clase)
 - Todos los objetos de un tipo particular pueden recibir los mismos mensajes
 - Alta cohesión y bajo acoplamiento
 - Alta cohesión: ser el máximo especialista en su área con las mínimas interferencias y solamente en su área
 - Bajo acoplamiento: La dependencia de objetos externos sea la mínima posible

CARACTERÍSTICAS DE LOS LENGUAJES OO

- Objetos
 - Empaquetan los datos (ESTADO) y la funcionalidad conjuntamente (COMPORTAMIENTO)
- Abstracción
 - La habilidad de un programa para ignorar ciertos aspectos de la información que está manipulando
- Encapsulación
 - Asegurar que los usuarios de un objeto no pueden cambiar su información o estado de formas no permitidas
- Polimorfismo
 - Diferentes objetos pueden tener la misma interfaz para responder al mismo tipo de mensaje, y hacerlo apropiadamente según su naturaleza
- Herencia
 - Permite a los objetos definirse como especializaciones de otros, que pueden compartir y extender su funcionalidad sin tener que reimplementarlo de nuevo

CLASES Y OBJETOS

- Todo objeto es una instancia de una clase, entendiendo ésta como la abstracción de un “tipo” de datos dentro del espacio del problema que a resolver
- Una clase describe un conjunto de objetos que tienen características idénticas (elementos de datos o atributos) y comportamientos idénticos (funcionalidades o métodos)

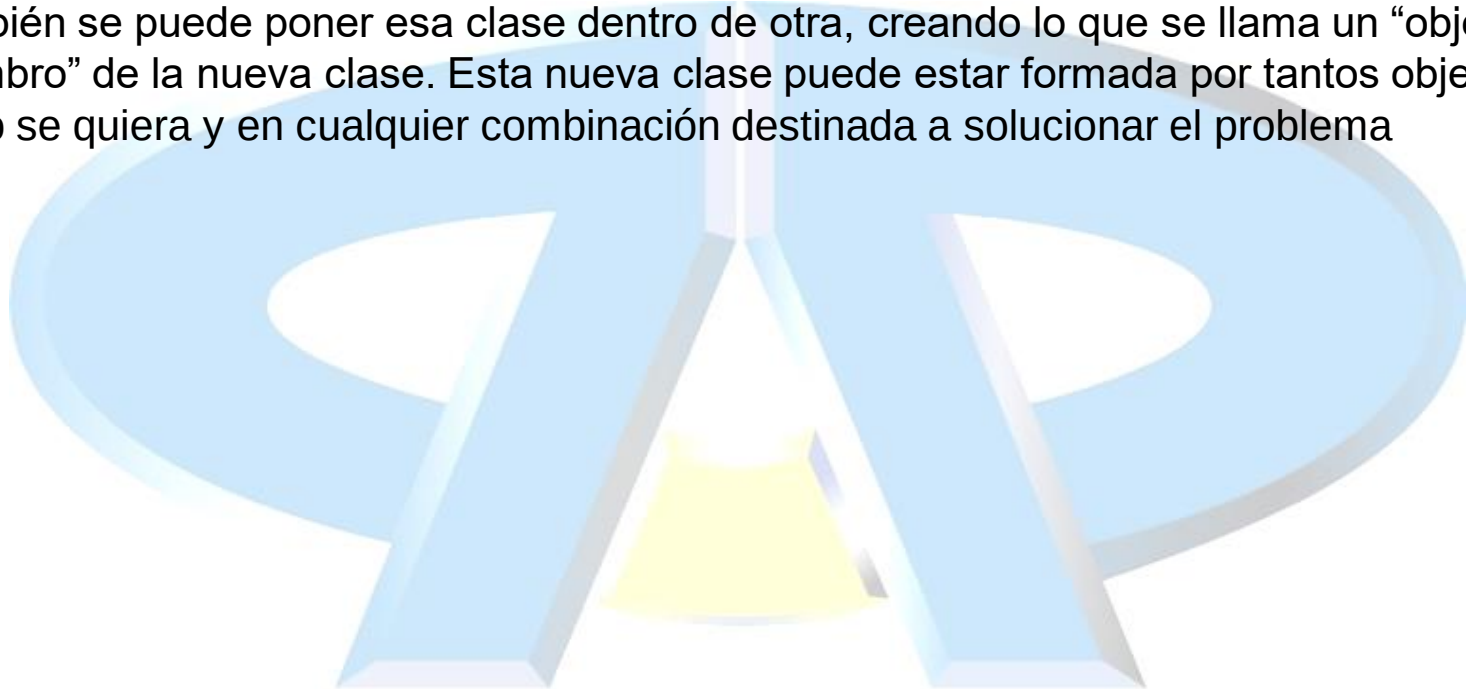


ENCAPSULACIÓN

- La encapsulación es la propiedad de las clases y los mecanismos que existen en cada lenguaje de programación que permiten definir qué métodos o atributos son públicos, privados o tienen algún otro mecanismo de protección.
- Separando la interfaz (pública) de la implementación (privada) se facilitan también las tareas de reescritura de procesos internos de la clase
 - *Mostrar lo que hago pero no como lo hago*

COMPOSICIÓN

- La forma más simple de utilizar una clase es instanciar un objeto de la misma
- También se puede poner esa clase dentro de otra, creando lo que se llama un “objeto miembro” de la nueva clase. Esta nueva clase puede estar formada por tantos objetos como se quiera y en cualquier combinación destinada a solucionar el problema

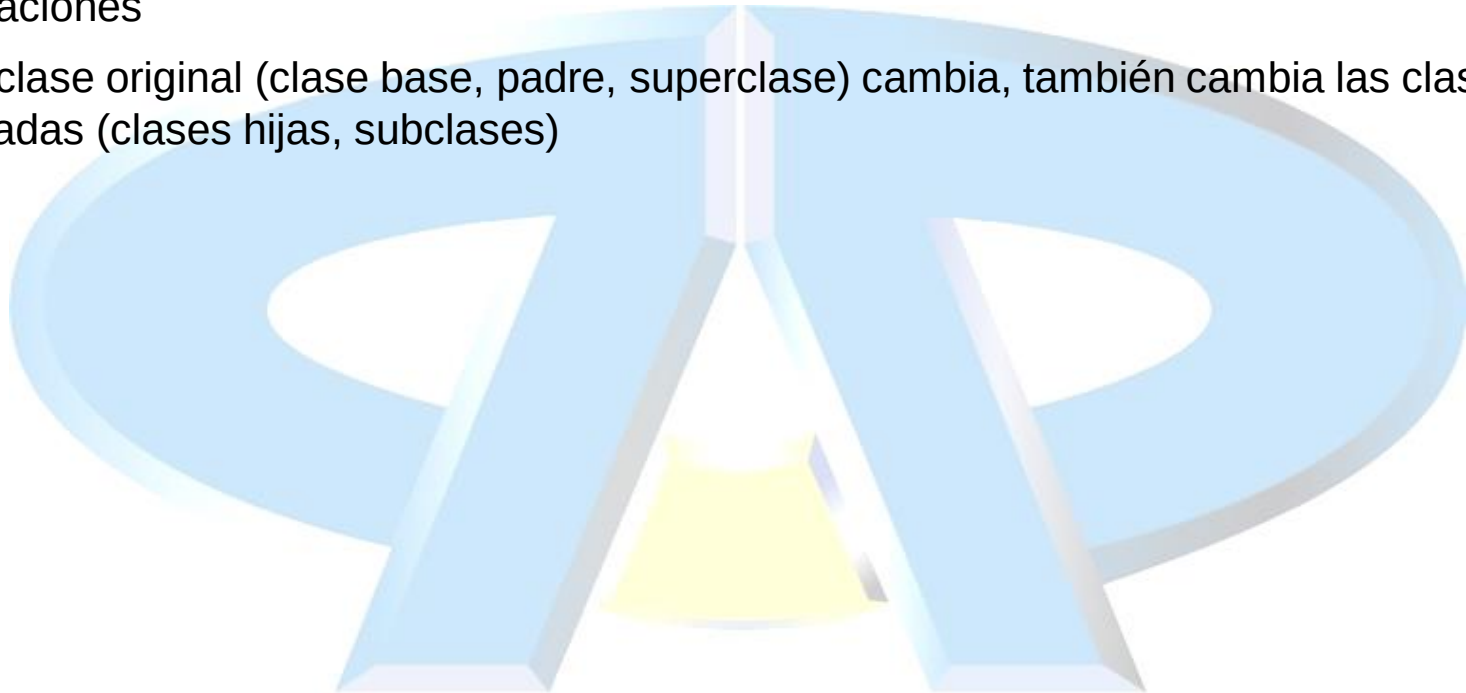


COMPOSICIÓN

- Por Valor
 - Es estática
 - El tiempo de vida del objeto incluido esta condicionado por el tiempo de vida del que lo incluye
 - Este tipo de relación es llamada **Composición** (el objeto base se construye a partir del objeto incluido, es decir, es "parte/todo")
- Por Referencia
 - Es dinámica
 - El tiempo de vida del objeto incluido es independiente del que lo incluye
 - Este tipo de relación es llamada **Agregación** (el objeto base utiliza al incluido para su funcionamiento)

HERENCIA

- A partir de una clase existente, se crea una copia en la que añaden o modifican prestaciones
- Si la clase original (clase base, padre, superclase) cambia, también cambia las clases derivadas (clases hijas, subclases)



HISTORIA UML

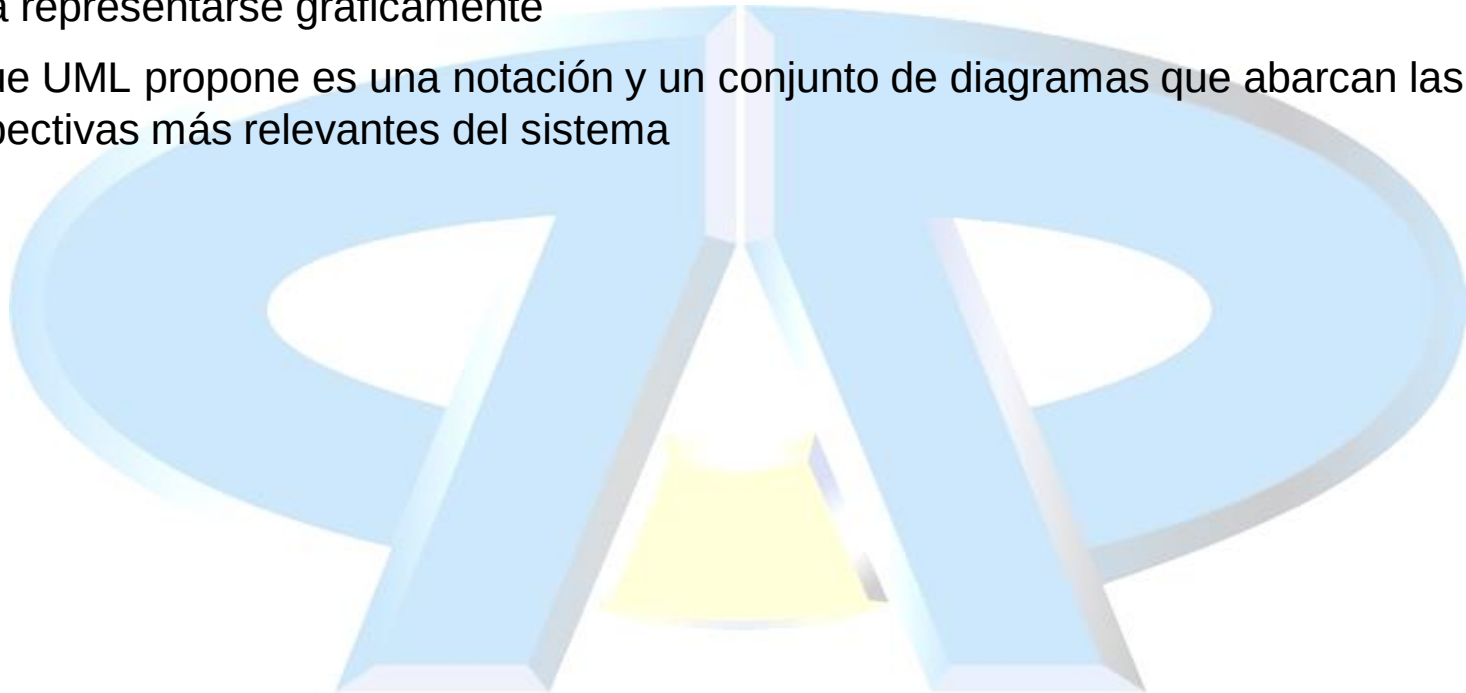
- A mediados de los años 90, la empresa Rational encargó a James Rumbaugh, Grady Booch, Ivar Jacobson la creación de un lenguaje de modelado
- UML (www.uml.org)
 - Unified Modeling Language
- Desde el año 2005 es un estándar ISO
- Impulsado por OMG (Object Management Group www.omg.org)

UML

- **MODELO**
 - Un modelo es una abstracción de un sistema o de un problema que hay que resolver considerando un cierto propósito o un punto de vista determinado
 - El modelo debe describir completamente los aspectos del sistema que son relevantes a su propósito y bajo un determinado nivel de detalle
- **DIAGRAMA**
 - Es una representación gráfica de una colección de elementos de modelado, a menudo dibujada como un grafo con vértices conectados por arcos

UML

- Cada punto de vista del sistema (y cada nivel de detalle) podrá modelarse y ese modelo podrá representarse gráficamente
- Lo que UML propone es una notación y un conjunto de diagramas que abarcan las perspectivas más relevantes del sistema



DIAGRAMAS UML

- Diagrama de casos de uso
- Diagrama de clases
- Diagramas de comportamiento
 - Diagrama de estados
 - Diagrama de actividad
 - Diagramas de interacción
 - Diagrama de secuencia
 - Diagrama de colaboración
- Diagramas de implementación
 - Diagrama de componentes



ELEMENTOS COMUNES

- Comentario / notas
 - Representan una anotación con motivos explicativos



CORRESPONDENCIA

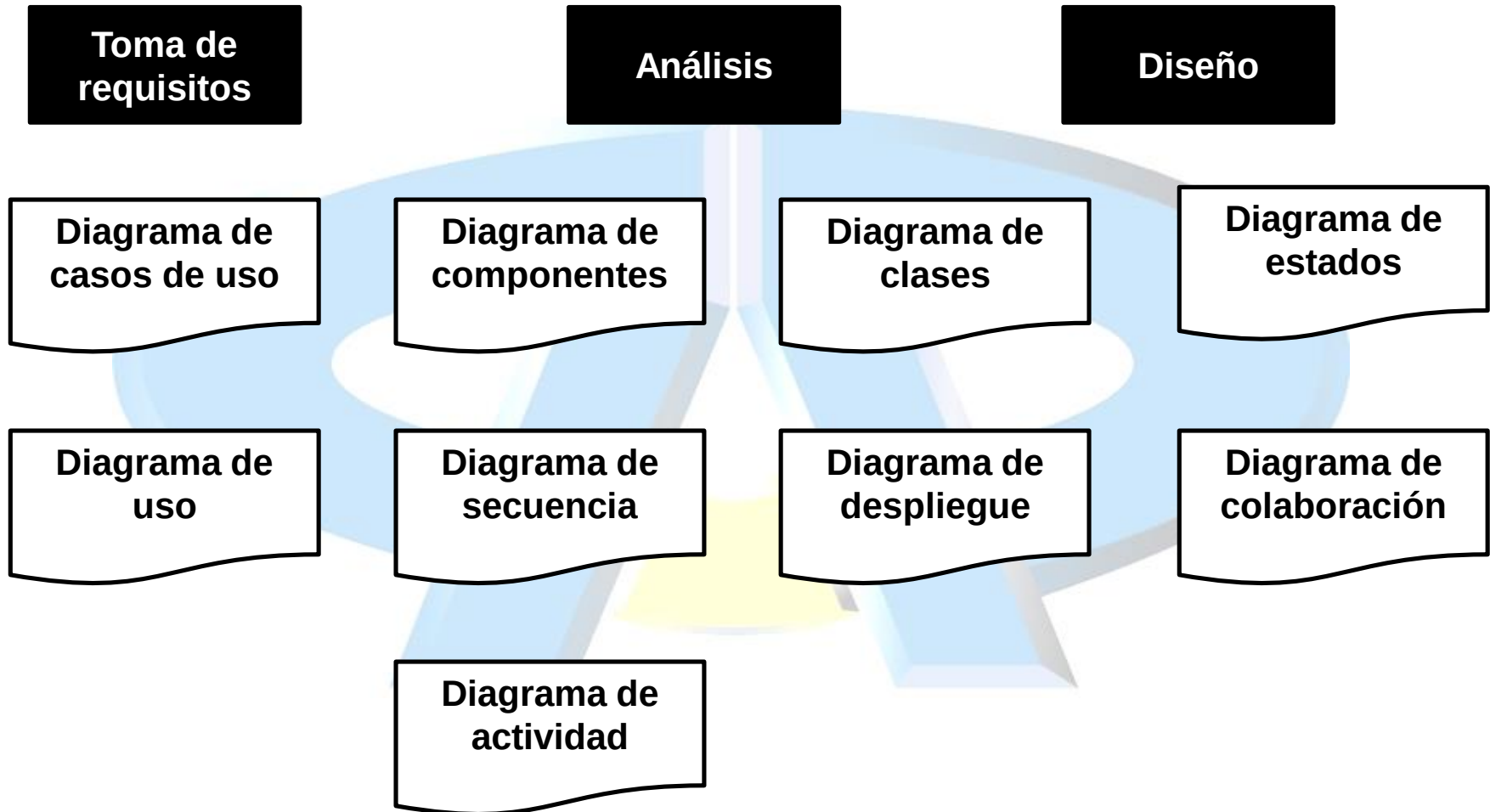




DIAGRAMA DE CASOS DE USO

CARACTERÍSTICAS

- Los casos de uso son una herramienta esencial en la toma de requisitos del sistema
- Permiten expresar gráficamente las relaciones entre los diferentes usos del mismo y sus participantes o actores
- No definen todos los requisitos (por ejemplo, tipos de datos, interfaces externas, rendimiento,...) pero sí que representan el hilo conductor que los vincula a todos con los actores del sistema.

ELEMENTOS

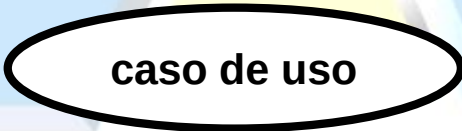
- **Actor**

- Es un rol que un usuario u otro sistema juega con respecto al sistema del problema
- Identificar a los actores de un caso de uso pasa por averiguar quién está involucrado en cada requisito concreto, quién se beneficiará de cada funcionalidad o quién proveerá o usará la información



- **Caso de uso**

- Son las acciones que pueden tener lugar en el sistema que queremos modelar
- Para identificarlos, puede ser útil preguntarse cuáles son las tareas y responsabilidades de cada actor, si habrá actores que recibirán información del sistema,...

A black UML Use Case symbol, consisting of an oval with the text "caso de uso" inside.

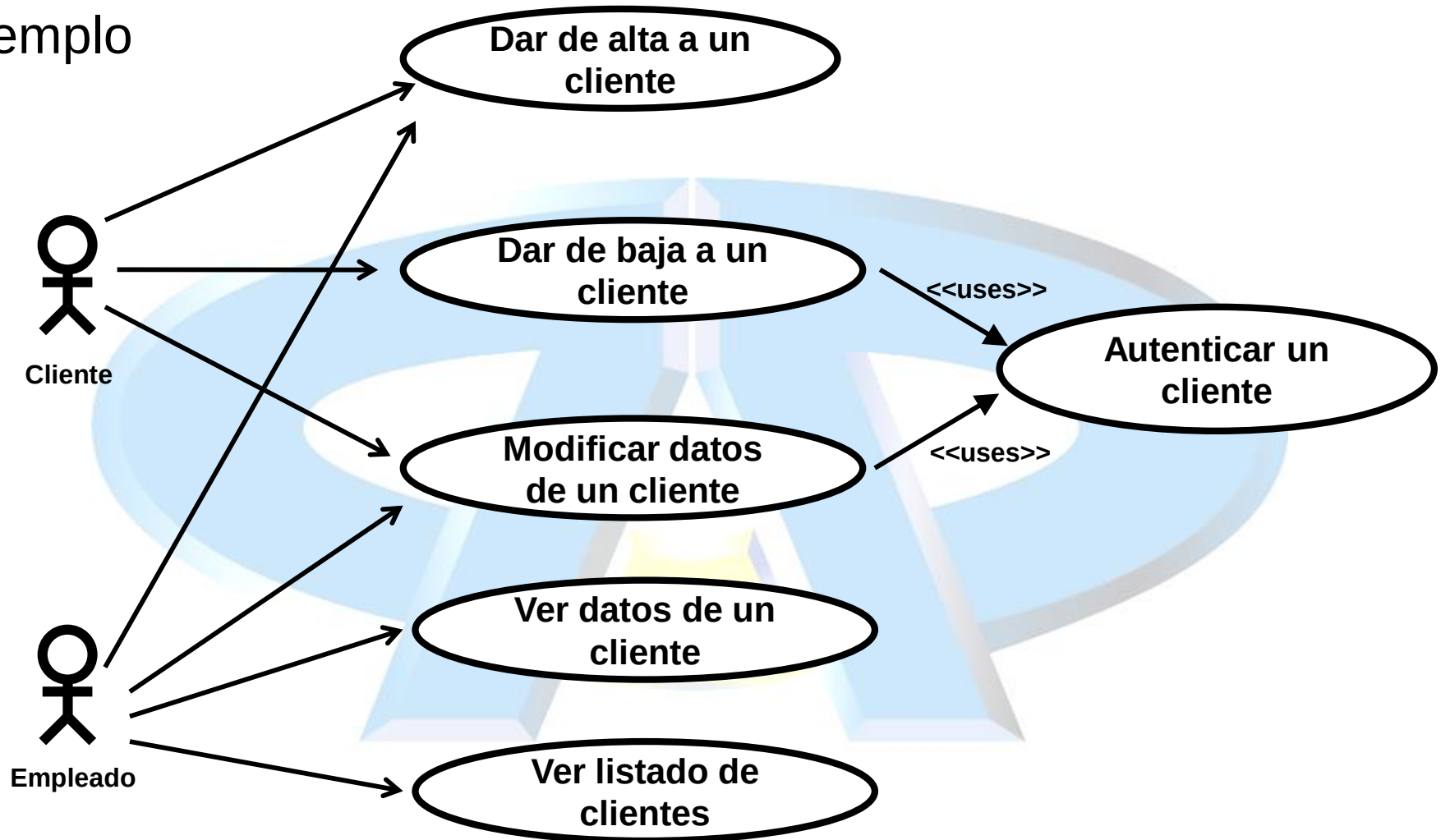
caso de uso

ELEMENTOS

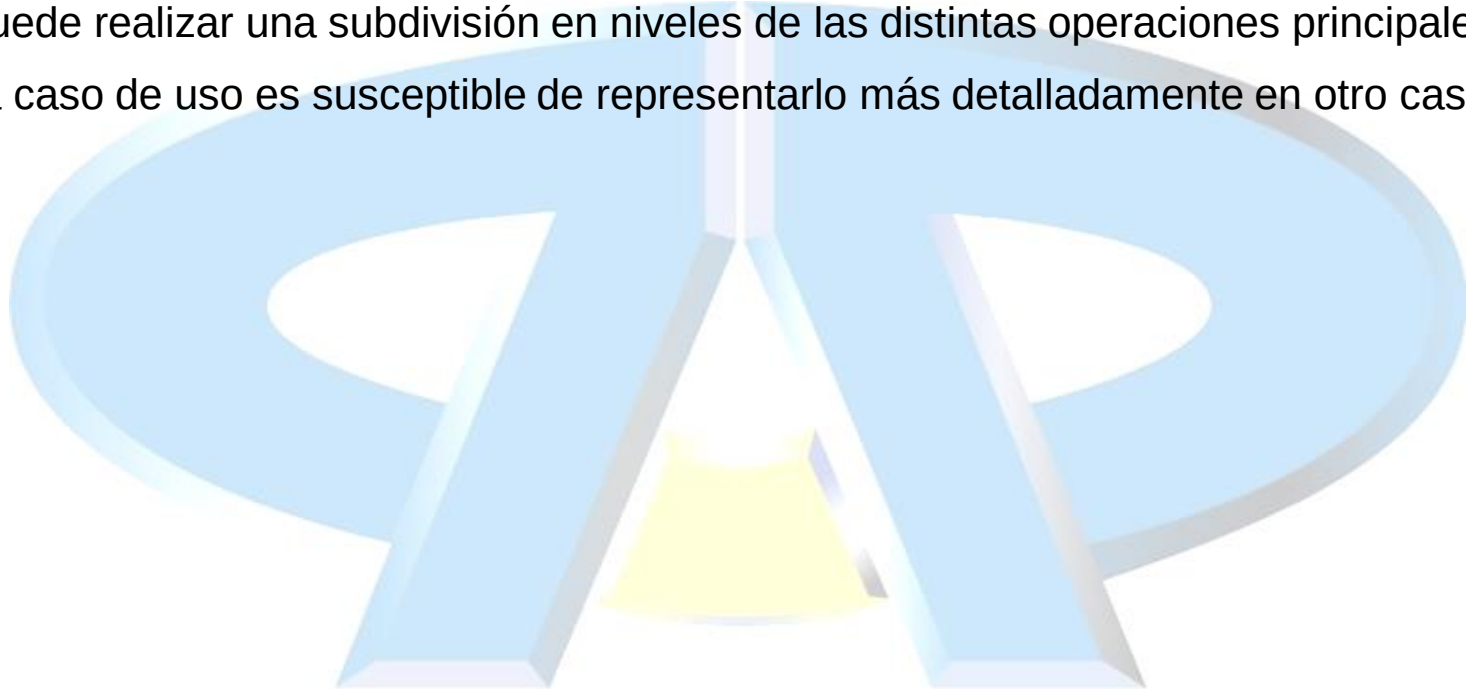
- Relaciones
 - **Asociación**
 - Es el tipo de relación más básica que indica la invocación desde un actor o caso de uso a otra operación (caso de uso)
 - **Generalización**
 - Este tipo de relación esta orientado exclusivamente para casos de uso (y no para actores)
 - **<<extends>> (herencia)**: Se recomienda utilizar cuando un caso de uso es similar a otro (características).
 - **<<uses>> (uso)**: Se recomienda utilizar cuando se tiene un conjunto de características que son similares en más de un caso de uso y no se desea mantener copiada la descripción de la característica.



Ejemplo



- Los casos de uso no es recomendable sobrecargarlos demasiado
- Se puede realizar una subdivisión en niveles de las distintas operaciones principales
- Cada caso de uso es susceptible de representarlo más detalladamente en otro caso de uso



BUENAS PRÁCTICAS

- Empezar los nombres de los casos de uso con un verbo
- Aunque no hay forma de indicar el orden en que un actor aplicará los casos de uso, suele ser más intuitivo representarlos en orden descendente, situando los más importantes en la parte superior del diagrama
- Situar a los actores principales en la parte superior del diagrama
- Nombrar a los actores con sustantivos relacionados con las reglas de negocio, de acuerdo con los roles que representan
- Anteponer “Sistema” o <<sistema>> a los actores que sean procesos externos
- Para representar eventos que ocurren de forma programada, se puede introducir un actor de sistema “Tiempo” para modelar sus casos de uso
- La etiqueta <<uses>> no es obligatoria
 - Situar los casos incluidos a la derecha del caso que los incluye
- No debe abusarse de la etiqueta <<extends>>, ya que dificulta la comprensión del caso
 - Situar los casos que extienden debajo del caso padre
- Debe evitarse representar más de dos niveles de asociaciones de casos de uso en un mismo diagrama. No es una representación funcional



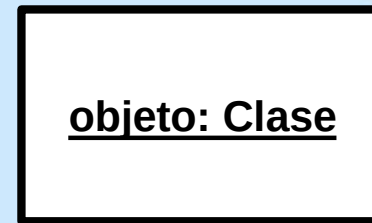
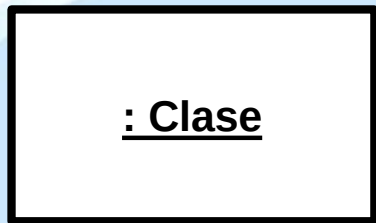
DIAGRAMA DE SECUENCIA

CARACTERÍSTICAS

- Los diagramas de secuencia modelan el flujo de la lógica dentro del sistema de forma visual, permitiendo documentarla y validarla
- Pueden usarse tanto en análisis como en diseño, proporcionando una buena base para identificar el comportamiento del sistema
- Típicamente se usan para modelar los casos de uso del sistema, describiendo de qué formas puede usarse
- La secuencia puede expresar tanto un caso de uso completo como quizá un caso concreto del mismo contemplando algunas alternativas

ELEMENTOS

- Objeto
 - Instancia de una clase que se puede empezar a identificar como participante en la secuencia de operaciones que representa este caso de uso



- Actores
 - Se relacionan con el caso de uso que se modela en la secuencia
 - Se relacionan con los objetos de la secuencia



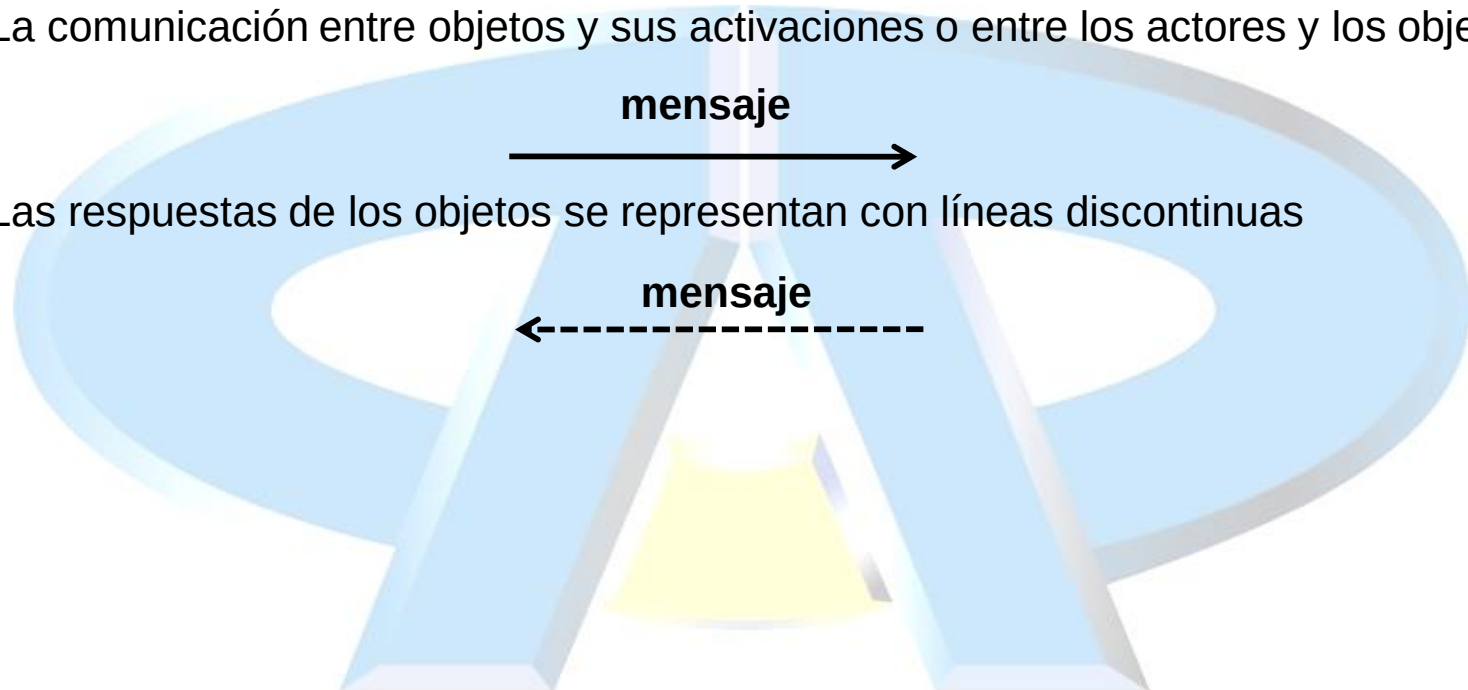
ELEMENTOS

- Vida del objeto
 - Se indica la existencia de un objeto en el caso de uso
- Activación
 - Se indica el instante del tiempo de vida, en el que el objeto está activo realizando una tarea
- Destrucción
 - Se indica el instante en el que el objeto ya no es necesario en el sistema



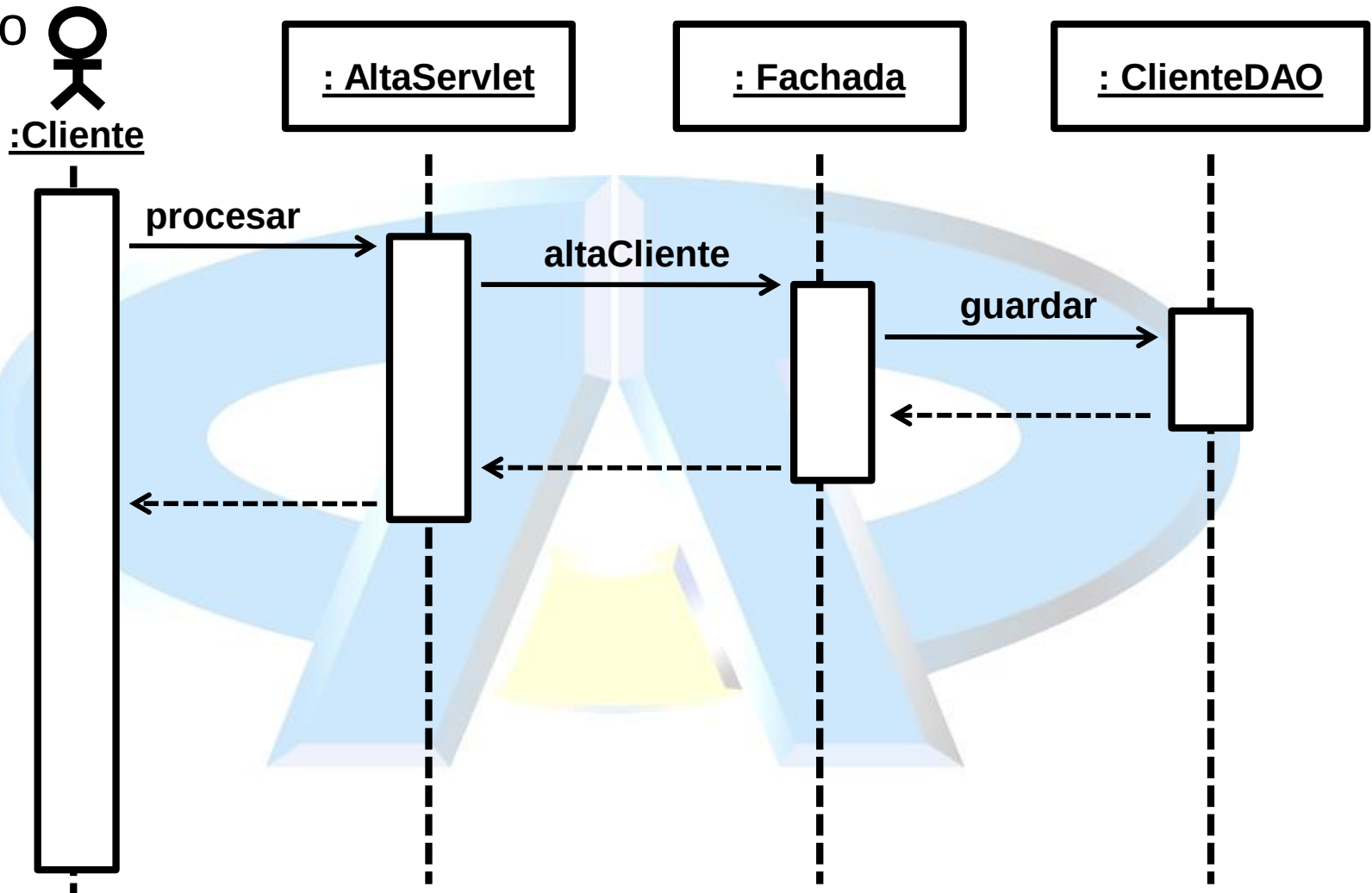
ELEMENTOS

- Mensaje
 - La comunicación entre objetos y sus activaciones o entre los actores y los objetos



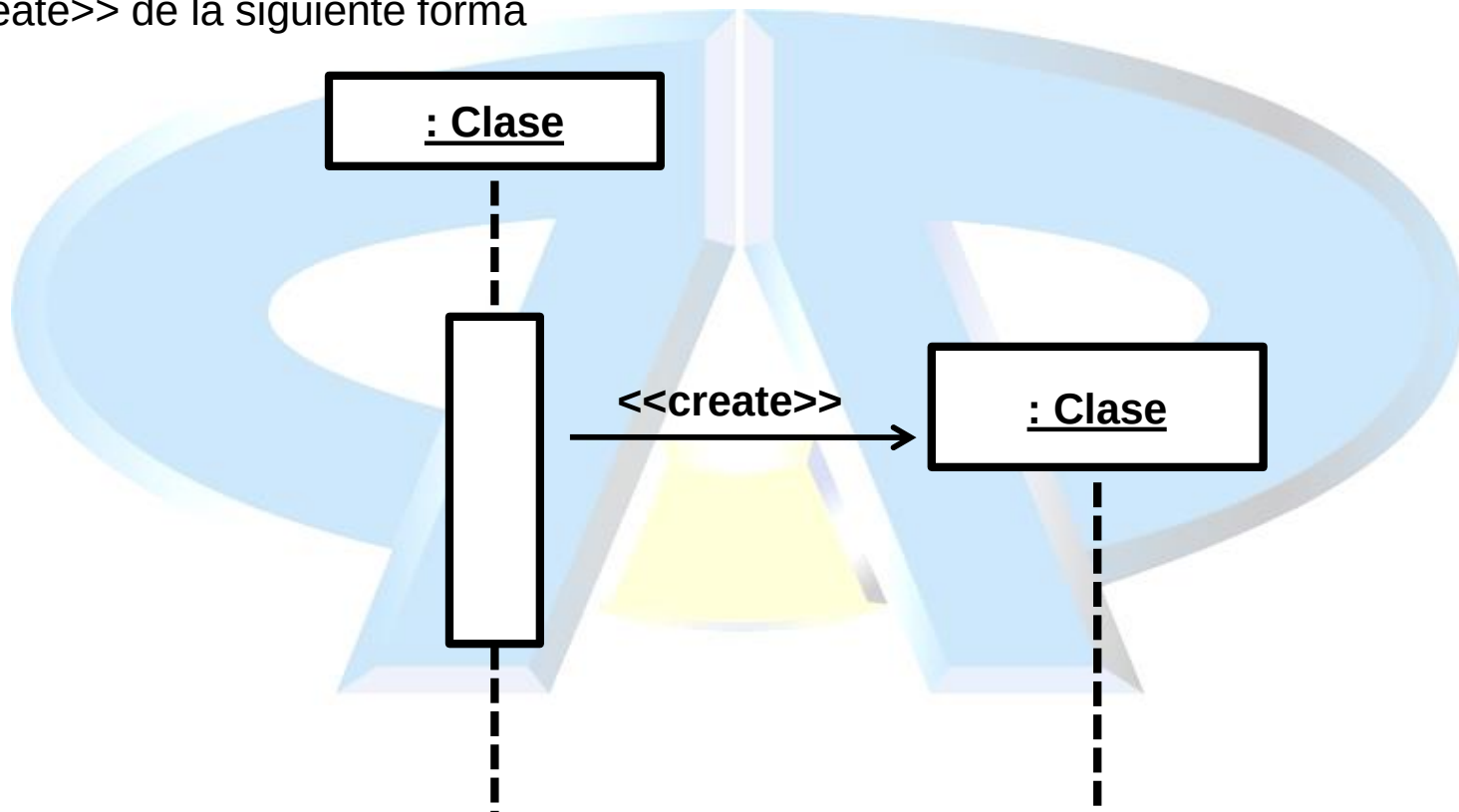
- Las respuestas de los objetos se representan con líneas discontinuas

Ejemplo

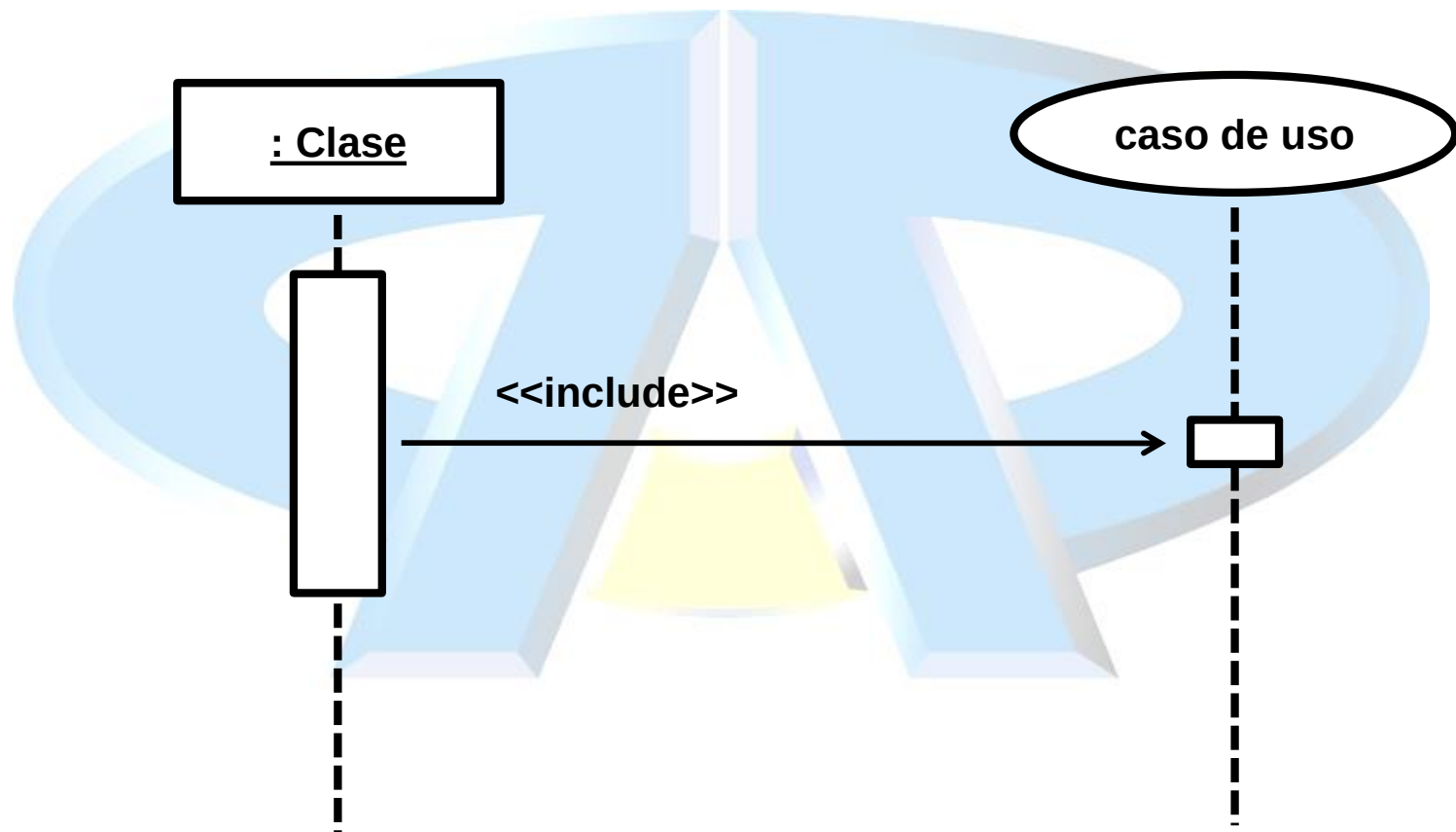


- En la parte superior, aparecen los participantes en la secuencia
- En general suele ser un actor el que inicie la secuencia
- Cada participante tiene un intervalo en el que éste está activo en la secuencia. El intervalo empieza cuando el objeto recibe un mensaje y termina cuando éste devuelve su última respuesta
- Los mensajes pueden incluir números de secuencia que facilitan la comprensión del diagrama y el seguimiento del orden en que se producen los mensajes.
- A veces un error puede provocar una línea discontinua de retorno hasta el primer participante
- Se pueden utilizar parámetros en los mensajes

- Si como resultado de un mensaje se crea un objeto, se representará con el estereotipo `<<create>>` de la siguiente forma



- En las llamadas a casos de uso, usar el estereotipo <<include>>



BUENAS PRÁCTICAS

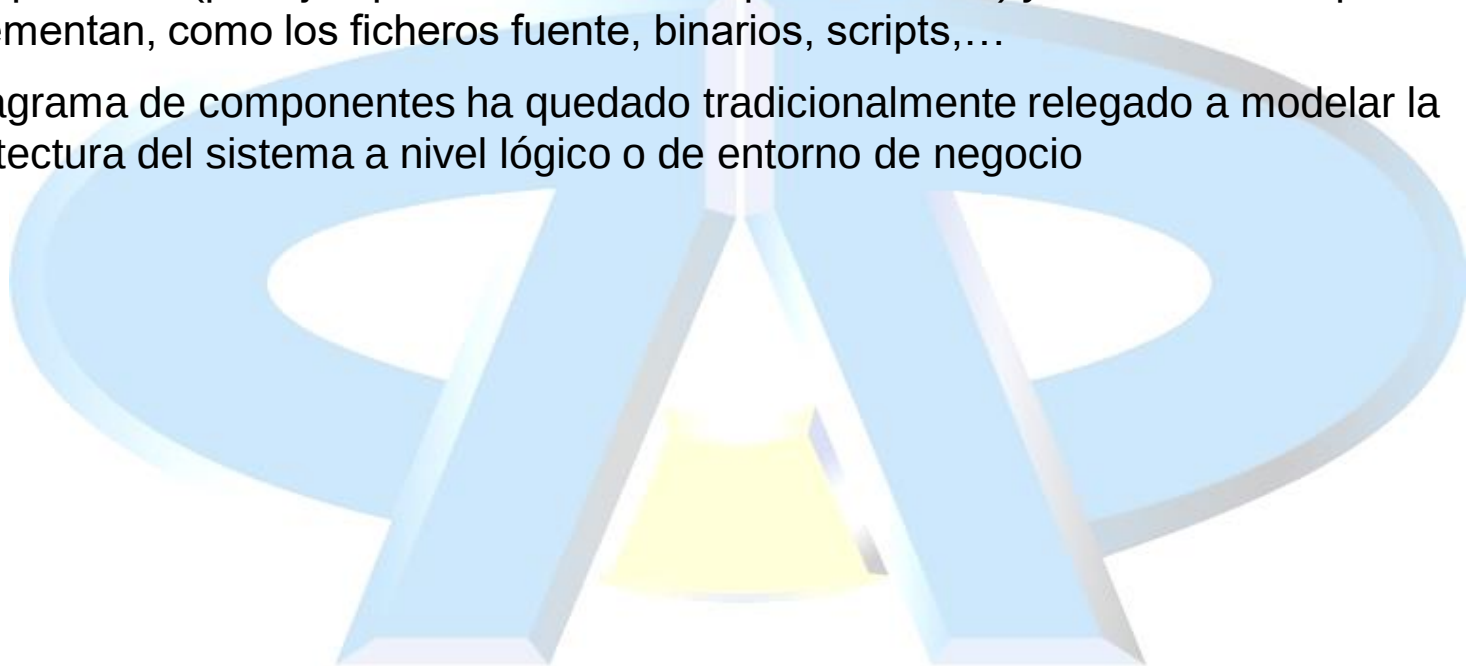
- El orden entre los mensajes y los participantes debe ser siempre de izquierda a derecha y de arriba abajo
- El nombre de los actores debe ser consistente con los casos de uso
- El nombre de los objetos debe ser consistente con los diagramas de clases
- Incluir notas en las secuencias
- Sólo incluir el aspa de destrucción del objeto en casos en que proporcione información sobre cuándo debe destruirse, en caso contrario “se ensuciará” el diagrama innecesariamente
- En los parámetros, es preferible usar nombres claros que los tipos de los mismos



DIAGRAMA DE COMPONENTES

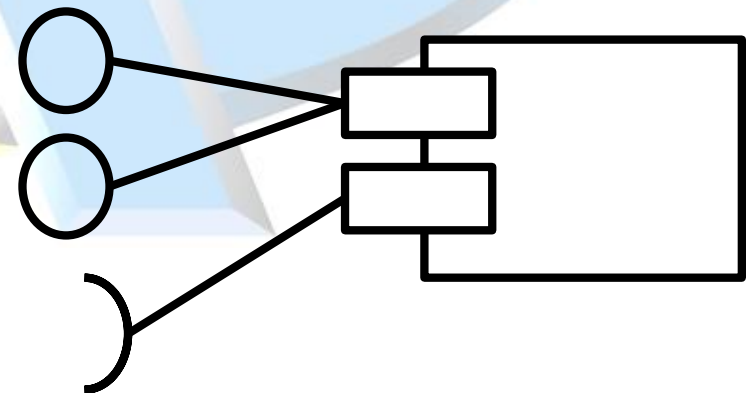
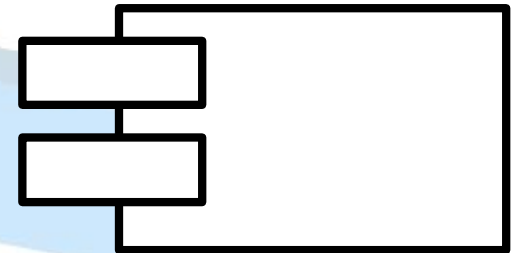
CARACTERÍSTICAS

- Muestra las dependencias entre componentes software, incluyendo los clasificadores que los especifican (por ejemplo las clases de implementación) y los artefactos que los implementan, como los ficheros fuente, binarios, scripts,...
- El diagrama de componentes ha quedado tradicionalmente relegado a modelar la arquitectura del sistema a nivel lógico o de entorno de negocio



ELEMENTOS

- Componente
 - Representa las piezas de software
 - Son bloques de construcción, que representan grandes porciones del sistema
 - Incorporan puertos hacia los datos o servicios que proporcionan
- Interfaces
 - Los componentes presentan interfaces hacia el exterior
 - Las interfaces representan algo que el componente proporciona (cerrado) o que necesita (abierto)



BUENAS PRÁCTICAS

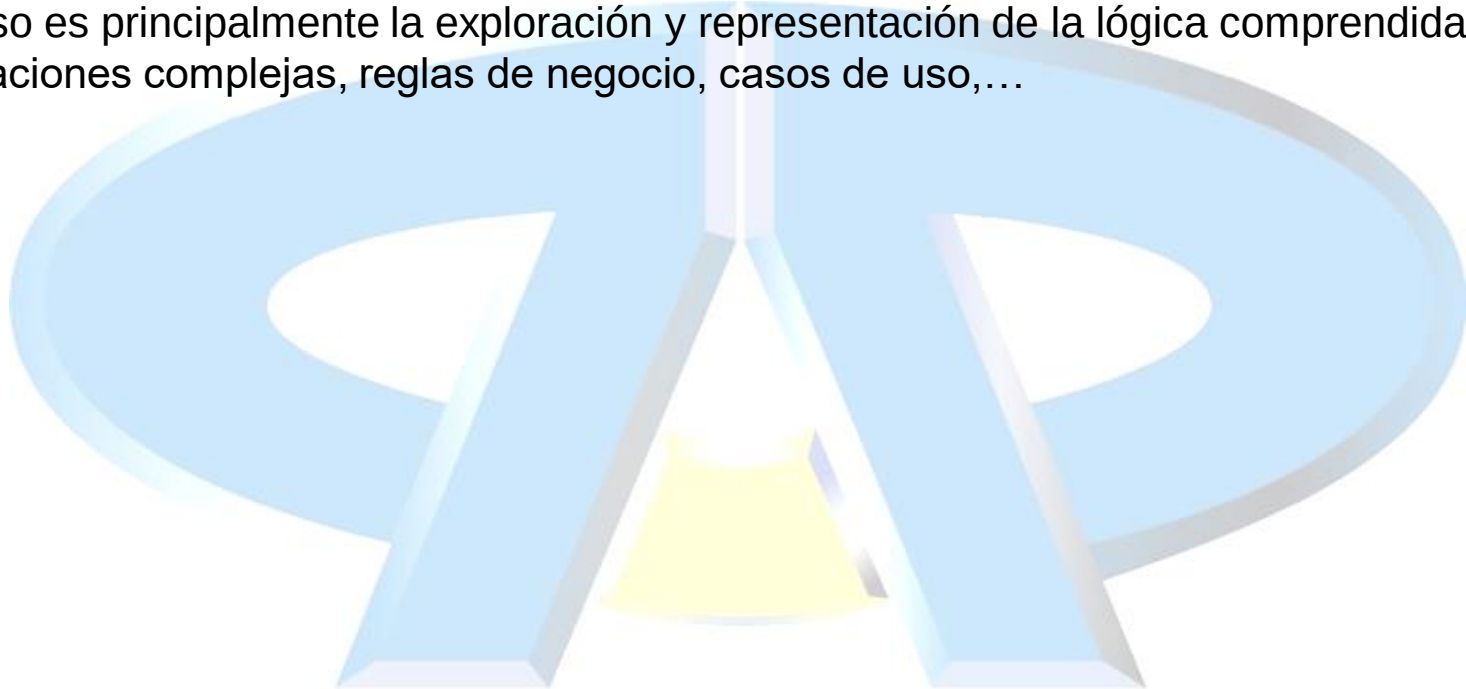
- Aplicar los estereotipos de forma consistente <<aplicación>>, <<bd>>, <<gui>>,...
- Ya que la interfaz es un conjunto de métodos, intentemos no representar demasiadas interfaces, es aconsejable agruparlas bajo un mismo nombre y crear después un diagrama detallado del componente
- Los componentes pueden heredar unos de otros. En este caso, la flecha que se utiliza en los casos de uso para expresar generalización puede servir perfectamente
- Para mejorar la comprensión de un diagrama de componentes, es preferible conectarlos siempre a través de interfaces. Es más consistente y evita confusiones o dudas sobre su interpretación



DIAGRAMA DE ACTIVIDAD

CARACTERÍSTICAS

- Son el equivalente orientado a objeto de los diagramas de flujo (DFD)
- Su uso es principalmente la exploración y representación de la lógica comprendida en operaciones complejas, reglas de negocio, casos de uso,...



ELEMENTOS

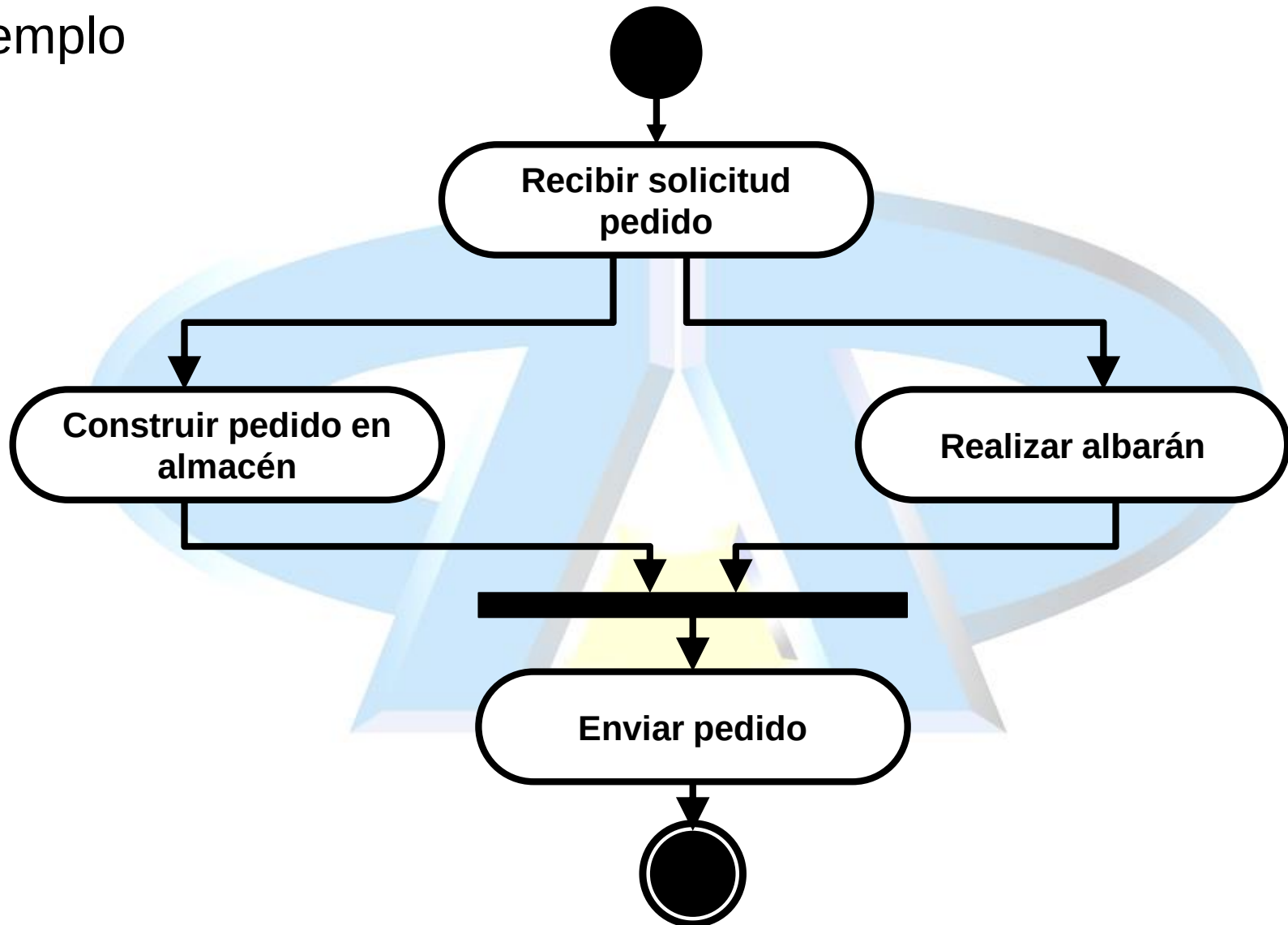
- Todo diagrama de actividades tiene un punto de partida y un punto final
- Actividad
 - Representarán cada paso importante que se produce en el proceso que estamos modelando
- Transición
 - Permite el paso de una actividad a otra
- Nodo de decisión
 - Permite elegir entre varios caminos
- Barra de sincronización
 - Punto de unificación. Permite paralelismo



BUENAS PRÁCTICAS

- Situar el punto de inicio en la parte superior del diagrama
- Aunque algunos autores lo consideran opcional (una actividad puede ser el final del diagrama), siempre es conveniente situar un punto de final de la actividad
- Si hay actividades que no tienen entrada pero sí salidas, o al revés, seguramente hay un error
- El rombo que indica una decisión no tiene la condición escrita dentro. Es en las transiciones que salen de él donde se escribe la condición evaluada y el resultado obtenido para seguir por esa transición
- Las salidas de los rombos de decisión deben contemplar todos los casos posibles

Ejemplo



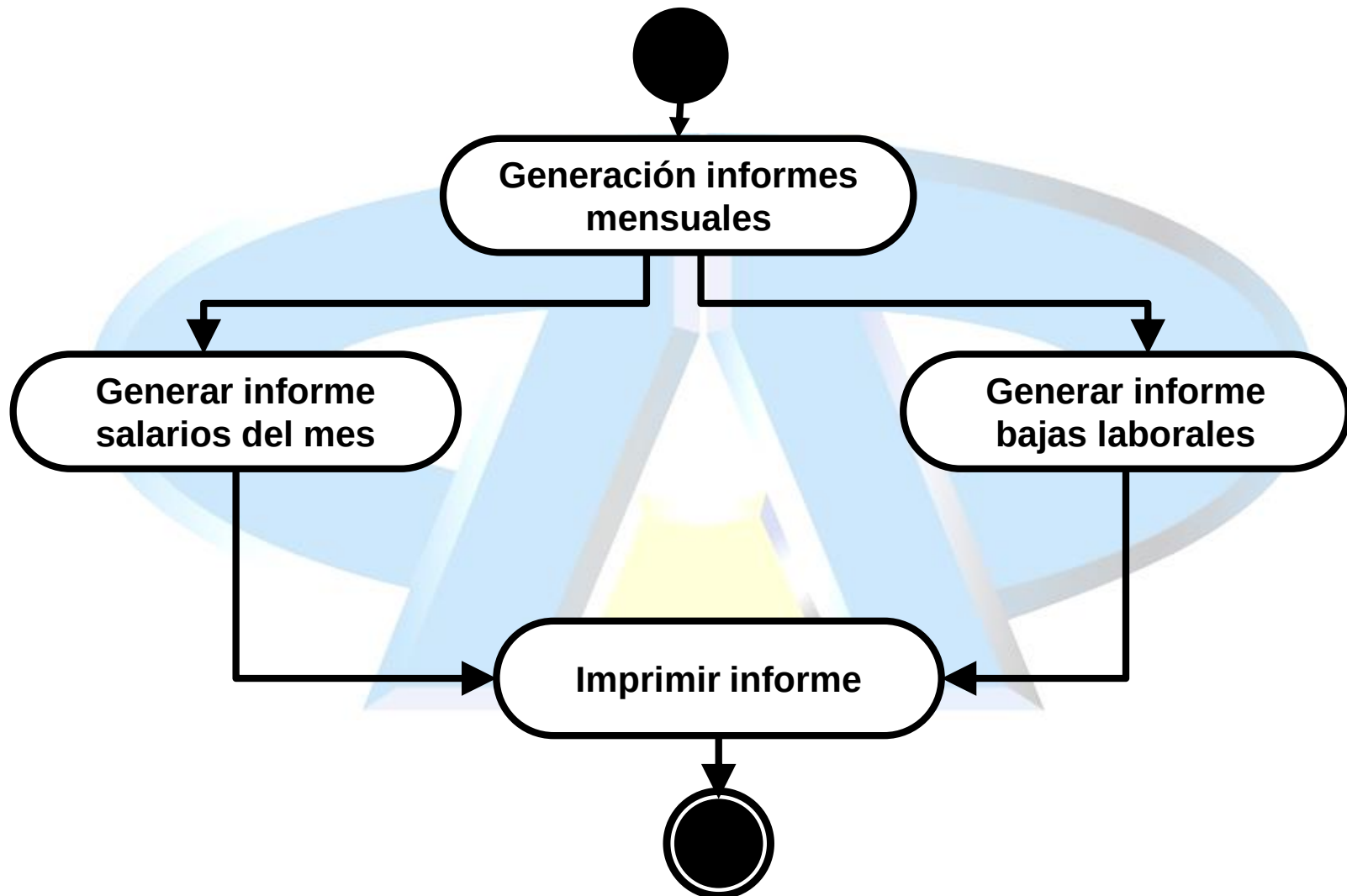
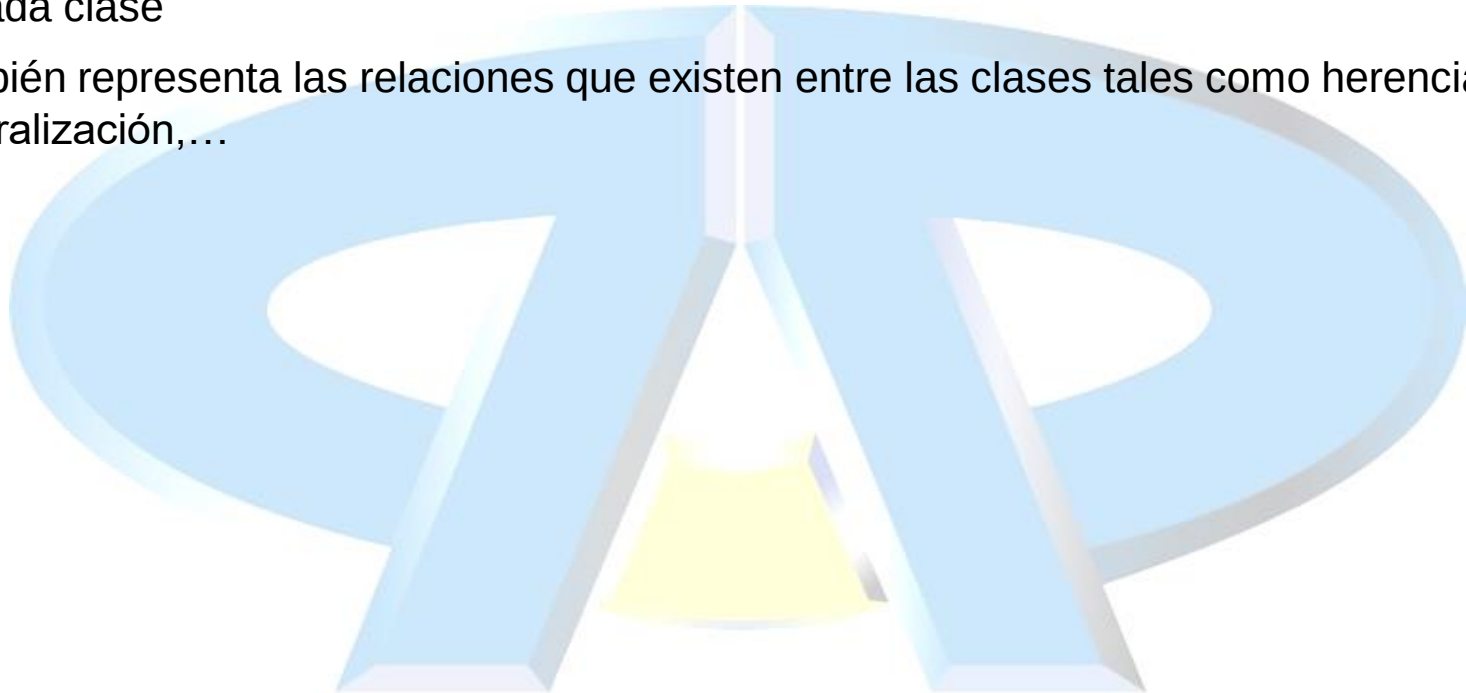




DIAGRAMA DE CLASES

CARACTERÍSTICAS

- Permite identificar la estructura de clases del sistema incluyendo las propiedades y métodos de cada clase
- También representa las relaciones que existen entre las clases tales como herencia, generalización,...



ELEMENTOS

- Clases
 - Representa una clase, dividido en tres partes
 - Nombre de la clase, y posibles estereotipos
 - Atributos
 - Lista de atributos con visibilidad y tipo
 - Métodos
 - Lista de métodos con visibilidad y tipo de retorno

<<estereotipo>>

Clase

- atributo : tipo

+ atributo : tipo

atributo : tipo

- método(param) : tipo

+ método(param) : tipo

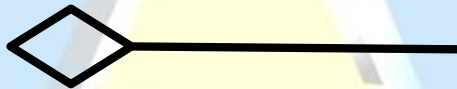
método(param) : tipo

ELEMENTOS

- Relaciones
 - Asociación
 - Relación simple, donde una clase referencia a otra
 - Dependencia
 - Relación, en la que la existencia de una clase, depende de otra
 - Las relaciones puede tener una navegabilidad, representada por la punta de flecha
 - También tienen una cardinalidad
 - 1, n, 1..*, 0..*

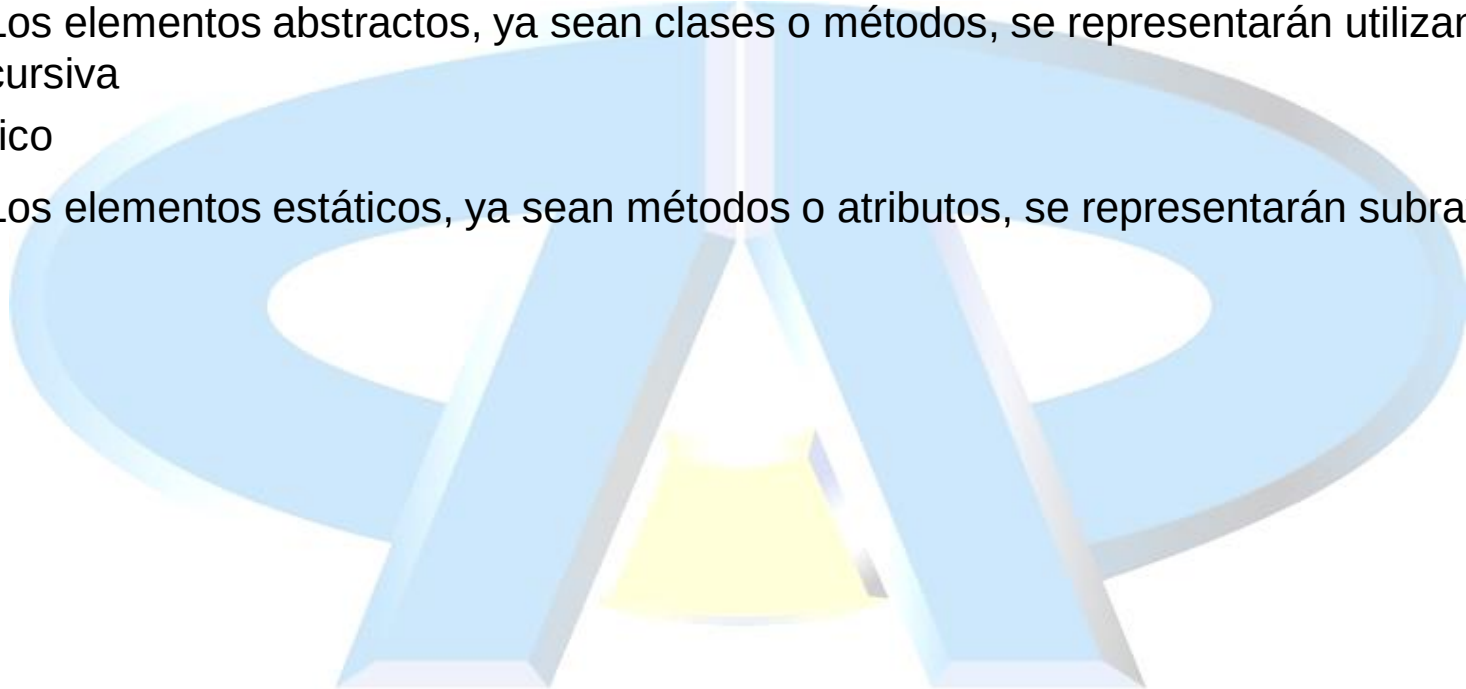
ELEMENTOS

- Relaciones
 - Herencia
 - Una clase hereda métodos y atributos de otra
 - Composición
 - Composición: depende para la existencia
 - Agregación: independencia



ELEMENTOS

- Abstracto
 - Los elementos abstractos, ya sean clases o métodos, se representarán utilizando letra cursiva
- Estático
 - Los elementos estáticos, ya sean métodos o atributos, se representarán subrayados



BUENAS PRÁCTICAS

- La visibilidad de los atributos (público: +; protegido: #; privado: -) es recomendable usarla sólo en fase de diseño
- Los nombres de los métodos y los atributos deberían reflejar las convenciones en cuanto a denominación del lenguaje de programación en que se vaya a implementar el sistema
- Cuando una asociación tiene métodos o atributos, deberemos modelarla como una clase que se asocia con las dos anteriores
- No es necesario incluir los métodos de acceso y modificación de los atributos (típicamente getXXX y setXXX)
- Si debemos dejar alguna lista de parámetros incompleta, indicarlo mediante una elipsis (...)
- Los atributos y operaciones estáticas deberían aparecer antes que las de instancia (el resto)

BUENAS PRÁCTICAS

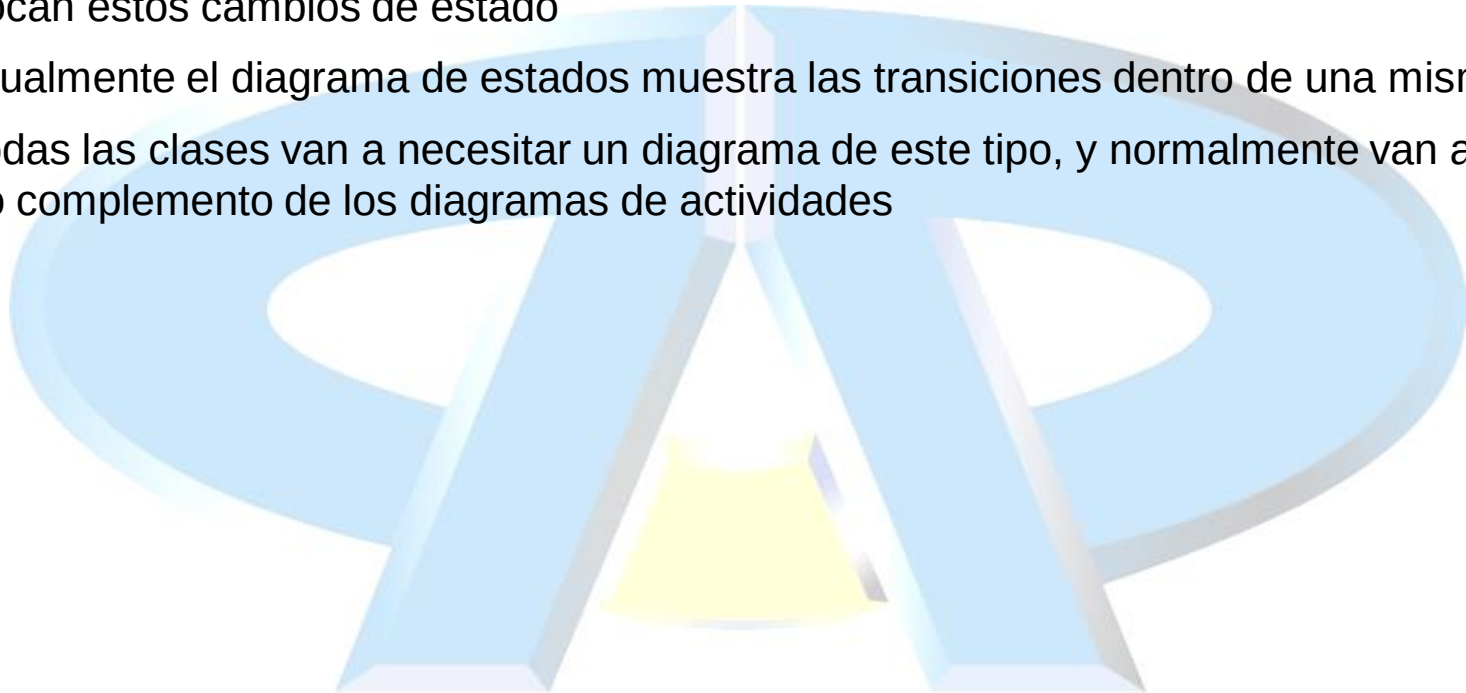
- Es conveniente ordenar los métodos y atributos según su visibilidad descendiente: públicos → protegidos → privados
- No conviene abusar de los estereotipos
- Si nuestro modelo incluye múltiples interfaces, puede ser útil incluirlos en otro diagrama y utilizar la notación para la clase que implemente la interfaz del diagrama de componentes en el diagrama de clases (el círculo cerrado)
- Se desaconseja repetir los atributos y métodos de la interfaz que implementa una clase
- Es aconsejable indicar siempre la multiplicidad de una relación
- Si existen varias relaciones entre dos clases, puede ser interesante indicar el rol que ocupan ellas en cada una



DIAGRAMA DE ESTADOS

CARACTERÍSTICAS

- Documentar los diferentes estados por los que una clase puede pasar y los eventos que provocan estos cambios de estado
- Habitualmente el diagrama de estados muestra las transiciones dentro de una misma clase
- No todas las clases van a necesitar un diagrama de este tipo, y normalmente van a usarse como complemento de los diagramas de actividades



ELEMENTOS

- Estado
 - Representa el estado de un objeto en un instante de tiempo
- Estado inicial y final
 - Pseudoestados (no tiene valor ni acciones) que representan el comienzo y el final

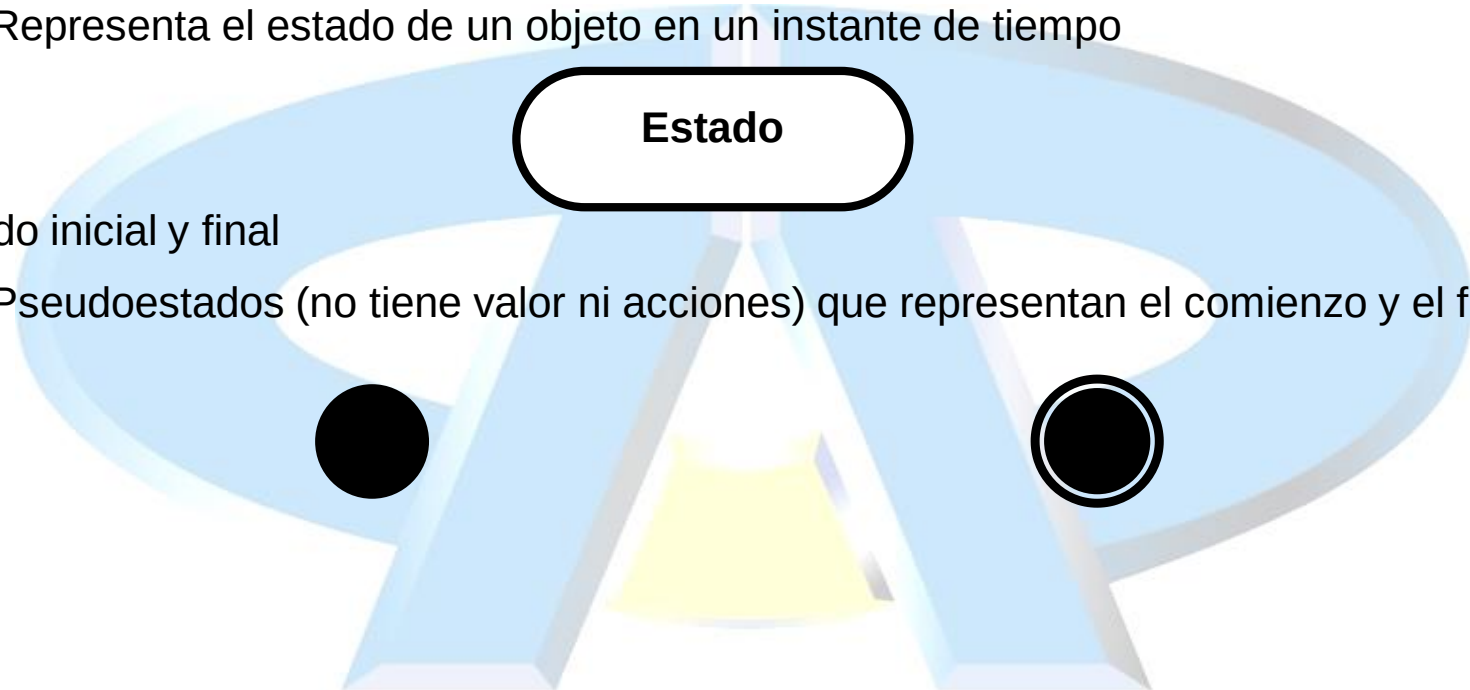
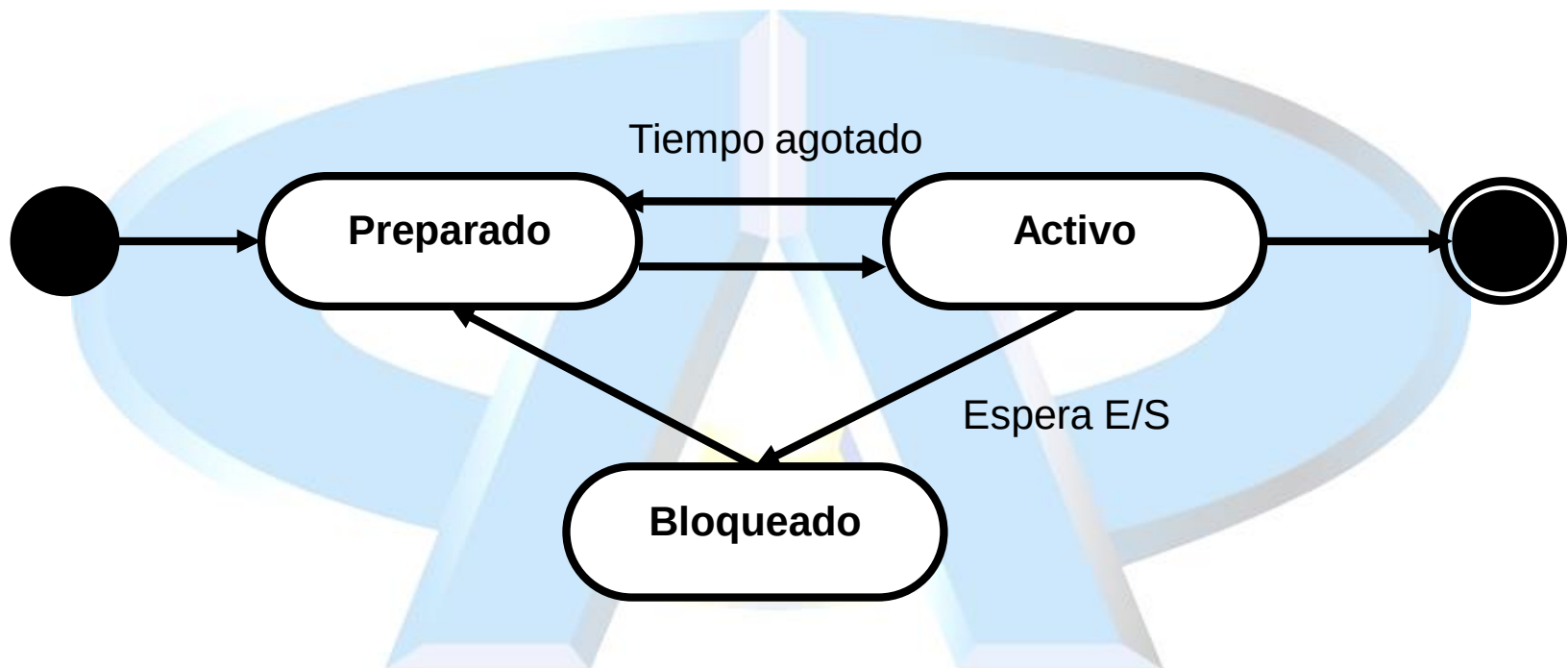
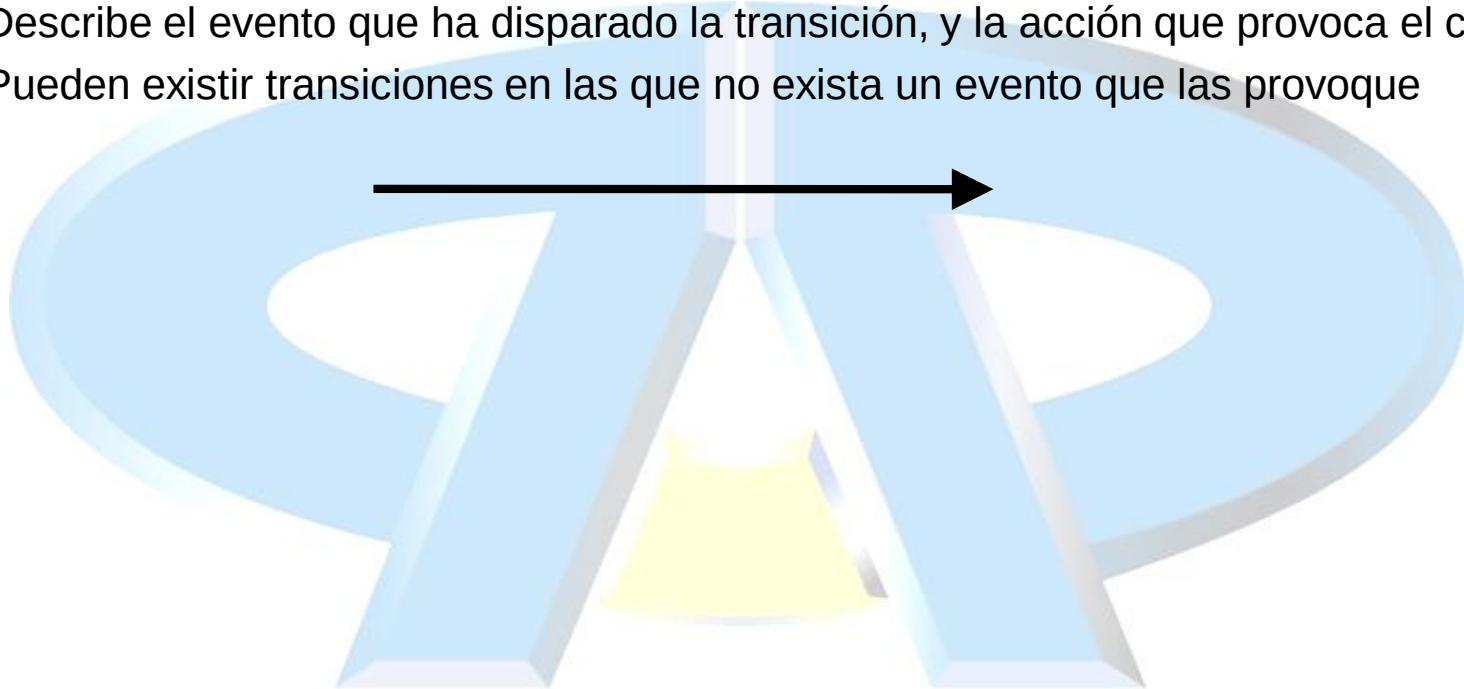


Diagrama de estados de un proceso



ELEMENTOS

- Transición
 - Describe el evento que ha disparado la transición, y la acción que provoca el cambio.
 - Pueden existir transiciones en las que no exista un evento que las provoque



BUENAS PRÁCTICAS

- Ordenar de arriba abajo los estados y sus transiciones (en concreto el estado inicial y final) ayuda a la comprensión del diagrama
- Para nombrar los estados, se aconseja usar verbos en presente (Actualizar, Presentar) o bien en participio (Actualizado, Presentado)
- Al igual que en los diagramas de actividad, habrá que vigilar con los estados que sólo emiten transiciones pero no provienen de ninguna, y de los estados que sólo reciben transiciones pero no expresan un cambio de estado del objeto
- Suele ser conveniente nombrar las acciones con los métodos que van a hacer cambiar el estado del objeto
- Si en un estado todas sus transiciones de salida o de entrada corresponden a la misma acción, y sólo varían en su condición, es correcto situar el nombre de la acción dentro del estado y sólo indicar la condición en cada transición
- De forma similar a los diagramas de actividad, deberemos repasar las condiciones de las transiciones para asegurarnos de que no hay un caso en que pueda haber dos transiciones posibles